

NumPy

The Complete Practical Guide

Arrays • Indexing • Broadcasting • Math & Linear Algebra • Statistics • Random Sampling • I/O •
Performance

A reference document with runnable code examples for every core NumPy feature.

Table of Contents

1. Introduction to NumPy
2. Installation and Import Convention
3. The ndarray Object
4. Creating Arrays
5. Array Attributes
6. Data Types (dtype)
7. Indexing and Slicing
8. Boolean and Fancy Indexing
9. Reshaping and Manipulating Arrays
10. Joining and Splitting Arrays
11. Arithmetic and Universal Functions (ufuncs)
12. Broadcasting
13. Aggregation and Statistical Functions
14. Sorting, Searching and Counting
15. Linear Algebra (numpy.linalg)
16. Random Sampling (numpy.random)
17. Set Operations
18. Copies vs Views
19. Vectorization and Performance
20. File I/O
21. Useful Miscellaneous Functions
22. Quick Reference Cheat Sheet

1. Introduction to NumPy

NumPy (Numerical Python) is the foundational package for numerical and scientific computing in Python. It provides a fast, memory-efficient multidimensional array object called **ndarray**, along with a large library of functions for operating on these arrays: mathematics, linear algebra, statistics, random number generation, Fourier transforms, and more.

Why NumPy is used so widely:

- **Speed:** Core operations are implemented in C, making them far faster than equivalent pure-Python loops.
- **Memory efficiency:** Arrays store homogeneous data in contiguous memory blocks, unlike Python lists.
- **Vectorization:** Operations apply to whole arrays at once, removing the need for explicit loops.
- **Broadcasting:** Arrays of different shapes can be combined in arithmetic operations without copying data.
- **Foundation for the ecosystem:** Pandas, SciPy, scikit-learn, TensorFlow, and PyTorch all build on NumPy arrays or its conventions.

2. Installation and Import Convention

Install NumPy with pip or conda:

```
pip install numpy

# or, with conda
conda install numpy
```

The universal convention is to import NumPy under the alias **np**:

```
import numpy as np

print(np.__version__) # check the installed version
```

3. The ndarray Object

The core data structure in NumPy is the **ndarray** (N-dimensional array). Every ndarray has a fixed size, a single data type (dtype) shared by all elements, and a shape describing its dimensions.

```
import numpy as np

a = np.array([1, 2, 3])
print(type(a))      # <class 'numpy.ndarray'>
print(a.ndim)       # 1 -> number of dimensions
print(a.shape)       # (3,) -> size along each dimension
print(a.dtype)       # int64 (platform dependent)
print(a.size)        # 3 -> total number of elements
```

Compared to a Python list, an ndarray is faster and uses less memory because it stores elements of the same type in one contiguous block, letting NumPy use optimized, compiled loops instead of Python's interpreter loop for each element.

4. Creating Arrays

4.1 From Python Lists / Tuples

```
a = np.array([1, 2, 3])          # 1-D array
b = np.array([[1, 2, 3], [4, 5, 6]]) # 2-D array (2 rows, 3 cols)
c = np.array([1, 2, 3], dtype=np.float32) # specify dtype explicitly
```

4.2 Built-in Array Constructors

```
np.zeros((2, 3))      # 2x3 array of zeros
np.ones((3, 3))       # 3x3 array of ones
np.full((2, 2), 7)    # 2x2 array filled with the value 7
np.empty((2, 3))      # uninitialized array (fast, garbage values)
np.eye(3)             # 3x3 identity matrix
np.identity(4)         # 4x4 identity matrix
np.diag([1, 2, 3])    # diagonal matrix from a 1-D array
```

4.3 Ranges and Sequences

```
np.arange(0, 10, 2)    # array([0, 2, 4, 6, 8]) (like range())
np.linspace(0, 1, 5)   # 5 evenly spaced values between 0 and 1
np.logspace(0, 3, 4)   # array([1, 10, 100, 1000]) log-spaced values
```

4.4 Random Arrays

```
np.random.rand(2, 3)   # uniform [0, 1), shape (2,3)
np.random.randn(2, 3)  # standard normal distribution
np.random.randint(0, 10, 5) # 5 random ints in [0, 10)
```

4.5 Array-like Templates

```
x = np.array([[1, 2], [3, 4]])
np.zeros_like(x) # zeros with same shape/dtype as x
np.ones_like(x)  # ones with same shape/dtype as x
np.full_like(x, 9) # filled with 9, same shape/dtype as x
```

5. Array Attributes

```
a = np.array([[1, 2, 3], [4, 5, 6]])

a.shape      # (2, 3)          - dimensions
a.ndim       # 2              - number of axes
a.size       # 6              - total elements
a.dtype      # dtype('int64') - element data type
a.itemsize   # 8              - bytes per element
a.nbytes     # 48             - total bytes consumed
a.T          # transpose of the array
```

6. Data Types (dtype)

NumPy supports many numeric types beyond Python's built-in int and float, including fixed-width integers, floats, complex numbers, and booleans.

```
np.array([1, 2, 3], dtype=np.int32)
np.array([1, 2, 3], dtype=np.float64)
np.array([1, 2, 3], dtype=np.bool_)

a = np.array([1.7, 2.3])
a.astype(np.int32) # cast to a different dtype -> array([1, 2])
```

Common dtypes: **int8/16/32/64**, **uint8/16/32/64**, **float16/32/64**, **complex64/128**, **bool_**.

7. Indexing and Slicing

```
a = np.array([10, 20, 30, 40, 50])
a[0]      # 10          - first element
a[-1]     # 50          - last element
a[1:4]    # [20 30 40]  - slice
a[:3]     # [10 20 30]
a[::2]    # [10 30 50]  - every second element
a[::-1]   # [50 40 30 20 10] - reversed

b = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
b[1, 2]   # 6          - row 1, col 2
b[0]      # [1 2 3]    - entire first row
b[:, 1]   # [2 5 8]    - entire second column
b[0:2, 0:2] # top-left 2x2 sub-matrix
```

8. Boolean and Fancy Indexing

8.1 Boolean Masking

```
a = np.array([1, 2, 3, 4, 5, 6])
mask = a > 3
print(mask)      # [False False False  True  True  True]
print(a[mask])   # [4 5 6]
a[a % 2 == 0] = 0 # set all even elements to 0
```

8.2 Fancy Indexing

```
a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
print(a[idx])     # [10 30 50]

b = np.array([[1, 2], [3, 4], [5, 6]])
print(b[[0, 2]])  # rows 0 and 2 -> [[1 2] [5 6]]
```

8.3 np.where

```
a = np.array([1, 2, 3, 4, 5])
np.where(a > 2, a, 0) # array([0, 0, 3, 4, 5])
np.where(a > 2)      # indices where condition is True: (array([2,3,4]),)
```

9. Reshaping and Manipulating Arrays

```
a = np.arange(12)
a.reshape(3, 4)      # reshape into 3x4 (data unchanged, new view where possible)
a.reshape(3, -1)     # -1 tells NumPy to infer that dimension
a.flatten()          # returns a 1-D copy
a.ravel()             # returns a 1-D view when possible (faster, no copy)
a.reshape(3,4).T     # transpose
np.expand_dims(a, axis=0) # add a new axis -> shape (1, 12)
np.squeeze(a.reshape(1,12)) # remove axes of length 1 -> shape (12,)
a.reshape(3,4).swapaxes(0,1) # swap two axes
```

10. Joining and Splitting Arrays

10.1 Joining

```

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.concatenate([a, b])      # [1 2 3 4 5 6]
np.stack([a, b])            # new axis -> [[1 2 3] [4 5 6]]
np.vstack([a, b])           # stack vertically (row-wise)
np.hstack([a, b])           # stack horizontally (column-wise for 1-D)
np.column_stack([a, b])     # stack as columns -> shape (3, 2)

```

10.2 Splitting

```

a = np.arange(9)
np.split(a, 3)              # 3 equal parts: [array([0,1,2]), array([3,4,5]), array([6,7,8])]
np.array_split(a, 4)        # split into 4 uneven parts if not evenly divisible

b = np.arange(16).reshape(4, 4)
np.hsplit(b, 2)             # split into 2 along columns
np.vsplit(b, 2)             # split into 2 along rows

```

11. Arithmetic and Universal Functions (ufuncs)

Arithmetic operators on arrays are applied **element-wise**. A ufunc (universal function) is a vectorized function that operates on ndarrays element by element, implemented in compiled C code for speed.

```

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

a + b      # [5 7 9]
a - b      # [-3 -3 -3]
a * b      # [4 10 18] (element-wise, NOT matrix multiplication)
a / b      # [0.25 0.4 0.5]
a ** 2     # [1 4 9]
a % 2      # [1 0 1]

np.add(a, b)
np.subtract(a, b)
np.multiply(a, b)
np.divide(a, b)
np.power(a, 2)
np.sqrt(a)      # element-wise square root
np.exp(a)       # element-wise e^x
np.log(a)       # natural log
np.log10(a)
np.abs(np.array([-1, -2, 3]))
np.sin(a); np.cos(a); np.tan(a)
np.round(np.array([1.234, 5.678]), 2) # [1.23 5.68]
np.floor(np.array([1.7, 2.3]))        # [1. 2.]
np.ceil(np.array([1.2, 2.8]))         # [2. 3.]

```

11.1 Matrix Multiplication

```

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

A @ B          # matrix product -> [[19 22] [43 50]]
np.matmul(A, B) # same as A @ B
np.dot(A, B)    # dot product (matrix product for 2-D arrays)

```

12. Broadcasting

Broadcasting lets NumPy perform arithmetic on arrays with different (but compatible) shapes, without explicitly copying data. Two dimensions are compatible when they are equal, or one of them is 1. NumPy compares shapes from the trailing (rightmost) dimension inward.

```

a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30])           # shape (3,)

print(a + b)
# b is broadcast across each row:
# [[11 22 33]
#  [14 25 36]]

c = np.array([[1], [2]]) # shape (2, 1)
d = np.array([10, 20, 30]) # shape (3,)
print(c + d)
# shapes (2,1) and (3,) broadcast to (2,3):
# [[11 21 31]
#  [12 22 32]]

```

A common, practical use is normalizing rows or columns of a matrix by subtracting a mean vector or dividing by a standard-deviation vector without writing a loop.

13. Aggregation and Statistical Functions

```

a = np.array([[1, 2, 3], [4, 5, 6]])

a.sum()           # 21           - sum of all elements
a.sum(axis=0)     # [5 7 9]      - sum down each column
a.sum(axis=1)     # [6 15]       - sum across each row
a.mean()          # 3.5          - average
a.std()           # standard deviation
a.var()           # variance
a.min(); a.max()  # 1, 6
a.argmin()        # index of min in flattened array
a.argmax()        # index of max in flattened array
a.cumsum()        # cumulative sum: [1 3 6 10 15 21]
a.cumprod()       # cumulative product
np.median(a)      # median value
np.percentile(a, 75) # 75th percentile
np.corrcoef(a[0], a[1]) # correlation coefficient matrix

```


Most aggregation functions accept an **axis** argument: axis=0 collapses rows (operates down columns), axis=1 collapses columns (operates across rows).

14. Sorting, Searching and Counting

```
a = np.array([3, 1, 4, 1, 5, 9, 2, 6])

np.sort(a)          # [1 1 2 3 4 5 6 9] (returns sorted copy)
a.sort()            # sorts in place
np.argsort(a)       # indices that would sort the array

b = np.array([1, 2, 3, 4, 5])
np.searchsorted(b, 3) # index where 3 should be inserted to keep order

c = np.array([1, 2, 2, 3, 3, 3])
np.unique(c)         # [1 2 3]
np.unique(c, return_counts=True) # (array([1,2,3]), array([1,2,3]))
np.count_nonzero(c == 3) # 3
np.any(c > 5)         # False
np.all(c > 0)         # True
```

15. Linear Algebra (numpy.linalg)

```
A = np.array([[4, 2], [1, 3]])

np.linalg.det(A)      # determinant
np.linalg.inv(A)      # matrix inverse
np.linalg.eig(A)      # eigenvalues & eigenvectors
np.linalg.matrix_rank(A) # rank of matrix
np.trace(A)           # sum of diagonal elements

b = np.array([1, 2])
np.linalg.solve(A, b)  # solve the linear system A x = b

np.linalg.norm(np.array([3, 4])) # Euclidean (L2) norm -> 5.0

U, S, Vt = np.linalg.svd(A) # singular value decomposition
Q, R = np.linalg.qr(A)      # QR decomposition
```

16. Random Sampling (numpy.random)

Modern NumPy code should prefer the **Generator** API (numpy.random.default_rng), which is faster and statistically more robust than the legacy np.random.* functions.

```

rng = np.random.default_rng(seed=42)  # reproducible generator

rng.random(5)                        # 5 floats in [0, 1)
rng.integers(0, 10, size=5)          # 5 random integers in [0, 10)
rng.normal(0, 1, size=5)             # 5 samples from N(mean=0, std=1)
rng.choice([1, 2, 3, 4], size=2, replace=False) # random sample w/o replacement
rng.shuffle(np.arange(5))            # shuffle in place

# Legacy (global) API:
np.random.seed(0)
np.random.rand(3)
np.random.randint(1, 100, 5)

```

17. Set Operations

```

a = np.array([1, 2, 3, 4])
b = np.array([3, 4, 5, 6])

np.union1d(a, b)      # [1 2 3 4 5 6]
np.intersect1d(a, b)  # [3 4]
np.setdiff1d(a, b)    # [1 2] - in a but not b
np.setxor1d(a, b)     # [1 2 5 6] - in exactly one of a, b
np.in1d(a, b)         # [False False True True]

```

18. Copies vs Views

Slicing a NumPy array returns a **view** (shares memory with the original) rather than a copy. Modifying a view modifies the original array. This is a common source of bugs, so use **.copy()** when an independent array is needed.

```

a = np.array([1, 2, 3, 4, 5])
b = a[1:4]      # view, not a copy
b[0] = 99
print(a)        # [ 1 99  3  4  5] <- original changed too!

c = a[1:4].copy() # explicit copy, independent memory
c[0] = -1
print(a)        # unchanged

# Fancy indexing and boolean masking ALWAYS return copies:
d = a[[0, 1, 2]] # copy
e = a[a > 2]     # copy

```

19. Vectorization and Performance

Replacing explicit Python for-loops with vectorized NumPy operations is usually the single biggest performance win available in numerical Python code.

```

import time

n = 1_000_000
a = list(range(n))
b = list(range(n))

# Pure Python loop
start = time.time()
c = [a[i] + b[i] for i in range(n)]
print('python loop:', time.time() - start)

# Vectorized NumPy
na = np.arange(n)
nb = np.arange(n)
start = time.time()
nc = na + nb
print('numpy vectorized:', time.time() - start)
# NumPy is typically 10-100x faster for this kind of operation

```

Other performance tips:

- Prefer built-in ufuncs (np.sum, np.sqrt, ...) over Python loops or np.vectorize.
- Avoid unnecessary .copy() calls; use views where safe.
- Choose the smallest dtype that fits your data range to save memory and bandwidth.
- Use out= parameters on ufuncs (e.g. np.add(a, b, out=a)) to avoid allocating new arrays.

20. File I/O

```

a = np.array([[1, 2, 3], [4, 5, 6]])

# NumPy's own binary format (fast, preserves dtype/shape exactly)
np.save('array.npy', a)
loaded = np.load('array.npy')

# Multiple arrays in one compressed file
np.savez('arrays.npz', x=a, y=a * 2)
data = np.load('arrays.npz')
data['x']; data['y']

# Plain text formats
np.savetxt('array.csv', a, delimiter=',')
loaded_csv = np.loadtxt('array.csv', delimiter=',')
genfromtxt_result = np.genfromtxt('array.csv', delimiter=',') # handles missing values

```

21. Useful Miscellaneous Functions

```

a = np.array([1, 2, 3])

np.clip(a, 1, 2)          # clamp values to [1, 2] -> [1 2 2]
np.tile(a, 2)             # repeat whole array -> [1 2 3 1 2 3]
np.repeat(a, 2)           # repeat each element -> [1 1 2 2 3 3]
np.pad(a, (1, 1))         # pad with zeros -> [0 1 2 3 0]
np.isnan(np.array([1, np.nan])) # [False True]
np.nan_to_num(np.array([1, np.nan])) # replace NaN with 0 -> [1. 0.]
np.meshgrid(np.arange(3), np.arange(2)) # coordinate grids
np.diff(np.array([1, 3, 6, 10])) # discrete difference -> [2 3 4]
np.gradient(np.array([1, 2, 4, 7])) # numerical gradient
np.allclose(np.array([1.0]), np.array([1.0000001])) # tolerant equality check
np.array_equal(a, np.array([1, 2, 3])) # exact equality

```

22. Quick Reference Cheat Sheet

Category	Function	Purpose
Create	np.array, np.zeros, np.ones, np.arange, np.linspace	Build arrays
Inspect	shape, ndim, size, dtype, itemsize	Array metadata
Index	a[i], a[i:j], a[mask], a[[i,j]]	Access elements
Reshape	reshape, ravel, flatten, transpose, expand_dims	Change layout
Combine	concatenate, stack, vstack, hstack, split	Join / split arrays
Math	+ - * / **, sqrt, exp, log, round	Element-wise ops
Matrix	@, np.dot, np.matmul, np.linalg.inv/det/solve	Linear algebra
Aggregate	sum, mean, std, var, min, max, median	Reduce along axis
Sort/Search	sort, argsort, unique, where, searchsorted	Order & lookup
Random	default_rng, .random, .integers, .normal, .choice	Random sampling
I/O	np.save/load, np.savez, np.savetxt/loadtxt	Persist arrays

End of guide — generated as a NumPy quick-reference with runnable code examples.